



Ministry of Communications and Information Technology (MCIT)

Digital Egypt Pioneers Initiative (DEPI) — Round 4

Frontend Development Track — React.js

Instructor: Mohamed Omar

Software Project Documentation

MedCare — Online Medical Appointment and Healthcare Information System

Frontend React.js Development Project

Team Members

Track	Frontend Development — React.js
Initiative	Digital Egypt Pioneers Initiative (DEPI) — Round 4
Organization	Ministry of Communications and Information Technology (MCIT)
Instructor	Mohamed Omar
Submission Date	April 2026
Document Version	2.0 — Enhanced Project Documentation

Team Members

#	Full Name	Role
1	Beshoy Saleh Gaber	Team Leader / Frontend Developer
2	George Arnest Rofaiel	Frontend Developer
3	Youssef Ashraf Darwish	Frontend Developer
4	Shady Sherif Salah	Frontend Developer

Revision History

Version	Date	Description	Author
1.0	April 14, 2026	Initial SRS — Phase 1 submission	Dev Team
2.0	April 25, 2026	Enhanced SRS: DEPI/MCIT branding, 5-member team, React.js stack, expanded FRs, NFRs, use cases, component architecture, and all 7 guideline sections	Dev Team

1. Project Planning & Management

1.1 Project Proposal

MedCare is a Medical Appointment Booking Web Application developed as part of the Digital Egypt Pioneers Initiative (DEPI) Round 4, under the Ministry of Communications and Information Technology (MCIT). The project is delivered as a React.js frontend application that enables patients to discover verified medical specialists, browse hospital services and departments, and schedule appointments through a modern, responsive, component-driven interface.

The application addresses a real-world need in Egyptian digital health services by providing citizens with a seamless, accessible online alternative to manual clinic booking. The system is aligned with MCIT's national digital transformation goals and the DEPI Frontend React.js Track curriculum requirements.

1.2 Project Plan — Timeline & Milestones

The project follows a structured six-week sprint plan aligned with the DEPI submission schedule:

Phase	Milestone	Deliverable	Start	End
1	Project Planning	Project Proposal, Gantt Chart, Risk Plan	Week 1	Week 1
2	Literature Review	Benchmark Analysis, Tech Stack Report	Week 2	Week 2
3	Requirements Gathering	SRS Document v1.0	Week 3	Week 3
4	System Analysis & Design	UML Diagrams, Component Architecture	Week 4	Week 4
5	Implementation	React.js Source Code, GitHub Repo	Week 5	Week 6
6	Testing & QA	Test Cases, Bug Reports	Week 6	Week 6
7	Final Presentation	SRS v2.0, Demo, PPT, User Manual	Week 7	Week 7

1.3 Task Assignment & Roles

Team Member	Role	Responsibilities
Beshoy Saleh Gaber (Team Leader)	Frontend Developer / Lead	Responsible for Home, Service and Service Details pages, as well as application routing and overall system integration, Responsible for Appointment, validation and API integration
George Arnest Rofaiel	Frontend Developer	Responsible for About, Department and Testimonials pages, including UI/UX implementation, Login and Sign Up pages

Team Member	Role	Responsibilities
Youssef Ashraf Darwish	Frontend Developer	Responsible for Doctor, Doctor Details and FAQ pages, including reusable components and dynamic UI
Shady Sherif Salah	Frontend Developer	Responsible for Contact Us, Blog and Blog Details pages, as well as content handling and testing

1.4 Risk Assessment & Mitigation Plan

#	Risk	Probability	Impact	Mitigation Strategy
1	API unavailability / mock data issues	Medium	High	Use JSON Server with static db.json; fallback to hardcoded mock data
2	Team member unavailability	Low	Medium	Document all tasks; cross-train on critical components; use GitHub issues
3	Scope creep beyond React.js frontend	Medium	Medium	Strictly limit to frontend; backend is simulated only
4	Performance issues on low-end devices	Medium	Medium	Code-split routes with React.lazy(); optimize images; Lighthouse audit
5	DEPI deadline not met	Low	High	Sprint planning with weekly check-ins; instructor feedback at each phase

1.5 KPIs (Key Performance Indicators)

#	KPI	Target Value	Measurement Method
1	Initial page load time (LCP)	≤ 2.5 seconds	Chrome Lighthouse audit
2	Page transition speed (SPA navigation)	≤ 1.5 seconds	Browser DevTools performance tab
3	System uptime during demo	99%	Manual observation during assessed demo
4	Appointment booking task completion time	≤ 5 minutes	Usability test with 3 evaluators
5	Mobile responsiveness coverage	100% of defined breakpoints	Chrome DevTools device emulation at 375px, 768px, 1440px

#	KPI	Target Value	Measurement Method
6	Zero ESLint errors in production build	0 lint errors	npm run lint before npm run build
7	Form validation coverage	100% of required fields validated	Manual test of all form fields with invalid inputs
8	User adoption rate (post-deployment)	≥ 80% task success rate	User acceptance testing sessions

2. Literature Review

2.1 Background and Domain Analysis

Medical appointment booking systems have evolved significantly from paper-based scheduling to sophisticated digital platforms. Modern healthcare booking solutions combine responsive frontend interfaces with dynamic data handling and basic patient authentication, and multi-specialty doctor profiles. Key characteristics observed across leading platforms in the domain include: dynamic appointment forms with service-to-doctor filtering, doctor profile pages with biography, qualifications, and working hours, department and specialty categorisation, patient testimonials for social proof, and integrated emergency contact information.

For the Egyptian healthcare context, digital transformation in healthcare aligns with MCIT's "Digital Egypt" strategy, which aims to digitise government and public services. A React.js frontend application for medical booking directly serves this objective by providing an accessible, mobile-responsive interface that citizens can use from any device.

2.2 Technology Review

The following technologies were evaluated and selected for the MedCare frontend project:

Technology	Purpose	Justification
React.js v18	Frontend framework	DEPI track requirement; industry-standard; component reusability; virtual DOM performance
React Router v6	Client-side routing	Standard routing for React SPAs; nested routes; data loaders
React Context API	Global state (auth, theme)	Lightweight; avoids Redux overhead for auth state
Axios	HTTP client	API communication with support for authentication tokens and structured error handling
Bootstrap 5	UI styling & responsiveness	Responsive grid; utility classes; matches medical UI design patterns
JSON Server	Mock REST API	Zero-config mock backend; enables frontend development without real server
Vite	Build tool	Faster HMR than CRA; modern ES modules; superior tree-shaking
React Hook Form	Form validation	Performant; minimal re-renders; easy integration with validation schemas

2.3 Comparative Analysis of Similar Systems

Platform	Key Features	Limitations	Lessons for MedCare
Vezeeta (Egypt)	Doctor search, booking, ratings, reviews, insurance filter	Complex UX; requires native app for full features	Simplified booking flow; prominent search; rating display
Zocdoc (USA)	Real-time availability, insurance check, teleconsultation	Not available in Egypt; complex integration	Dynamic slot display; service filtering by specialty
Medical website Templates	Service pages, doctor cards, appointment forms	Static; no real-time booking; no auth	Convert static UI to dynamic React state-driven components

2.4 Feedback & Evaluation Criteria (DEPI)

The DEPI instructor evaluation criteria for the Frontend React.js track project are:

- Component architecture quality: proper decomposition, reusability, and separation of concerns.
- State management: appropriate use of `useState`, `useEffect`, `useContext`, and custom hooks.
- Routing: correct implementation of React Router v6 with protected routes.
- Responsiveness: mobile-first design passing all breakpoint tests.
- Code quality: ESLint compliance, Prettier formatting, meaningful naming, `PropTypes` definitions.
- Functionality completeness: all core features (booking, doctor profiles, services, contact) implemented.
- Documentation quality: this project documentation assessed for completeness and technical accuracy.
- Code modularity and maintainability.

2.5 Suggested Improvements Over Previous Versions

- Version 1.0 lacked explicit React component mapping for each requirement — addressed in v2.0 with component columns in all requirement tables.
- Version 1.0 did not include a Technology Stack review — added in Section 2.2 above.
- Version 1.0 had no Gantt chart or KPIs — added in Section 1 above.
- Version 1.0 did not follow the DEPI documentation guidelines section order — fully corrected in v2.0.

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Role	Interest in System	Priority	Modules Affected
Patient / Visitor	Primary User	Finds doctors, books appointments, reads services & FAQ	High	All modules
Doctor	Displayed Entity	Profile displayed; working hours shown; appointment CTA	High	Doctor, Appointment
System Administrator	Content Manager	Manages system content (basic data management via mock or simple backend)	High	Frontend modules, basic admin interactions
Mohamed Omar	DEPI Instructor	Evaluates React.js system against DEPI Frontend Track criteria	High	All modules
MCIT / DEPI	Initiative Owner	Oversees deliverable quality for national digital transformation	High	All modules

3.2 User Stories & Use Cases Summary

User stories were derived from healthcare domain requirements and common user interactions in a modern medical booking system. Each story is written in the standard format: As a [role], I want to [goal], so that [benefit].

ID	User Story	Priority	React Component
US-01	As a visitor, I want to see a hero banner with a prominent "Get Appointment" button so that I can quickly initiate a booking.	Must Have	<HeroSlider>
US-02	As a patient, I want to register an account with my name, email, phone, and password so that I can access personalised features.	Must Have	<SignUpForm>
US-03	As a patient, I want to log in with my email and password so that I can access my account and basic booking information.	Must Have	<LoginForm>

ID	User Story	Priority	React Component
US-04	As a patient, I want to browse all doctors with their photos, specialty, and a booking CTA so that I can choose the right doctor.	Must Have	<DoctorCard>
US-05	As a patient, I want to view a full doctor profile with biography, education, and working hours so that I can make an informed decision.	Must Have	<DoctorProfile>
US-06	As a patient, I want to book an appointment by selecting a service, doctor, and providing my contact details so that I can submit an appointment request.	Must Have	<AppointmentForm>
US-07	As a patient, I want the doctor dropdown to filter based on the selected service so that I only see relevant doctors.	Must Have	<AppointmentForm>
US-08	As a patient, I want to view or manage appointment status (simulated interaction) so that I can free the slot if my plans change.	Must Have	<AppointmentHistory>
US-09	As a visitor, I want to browse all hospital services so that I can understand what treatments are available.	Must Have	<ServiceCard>
US-10	As a visitor, I want to see all hospital departments so that I can navigate to the right department.	Must Have	<DepartmentCard>
US-11	As a visitor, I want to read the FAQ so that I can find answers to common medical questions.	Should Have	<FAQAccordion>
US-12	As a visitor, I want to read blog articles about health topics so that I can stay informed.	Should Have	<BlogCard>
US-13	As a visitor, I want to contact the clinic by submitting a message form so that I can make enquiries.	Must Have	<ContactForm>
US-14	As a visitor, I want to subscribe to a newsletter so that I receive health updates.	Should Have	<NewsletterForm>
US-15	As a patient, I want to read testimonials from other patients so that I can trust the platform.	Should Have	<TestimonialCard>

3.3 Functional Requirements

A. Patient / Visitor Requirements

ID	Requirement Description	Priority	Status	React Component
FR-01	The system shall display a responsive Hero section with an auto-rotating slider and "Get Appointment" / "Learn More" CTA buttons.	Must Have	Planned	<HeroSlider>
FR-02	The system shall show an always-visible top bar displaying emergency phone numbers, email address, and clinic location.	Must Have	Planned	<TopBar>
FR-03	The system shall provide a Patient registration form capturing full name, email, phone, date of birth, and password with client-side validation.	Must Have	Planned	<SignUpForm>
FR-04	The system shall authenticate registered Patients via email and password, handling basic authentication logic.	Must Have	Planned	<LoginForm>, <AuthProvider>
FR-05	The system shall provide a logout function that clears user session data and redirects to the home page.	Must Have	Planned	<Navbar>
FR-06	The system shall render a Doctor Listing page showing all approved doctors as cards with photo, name, specialty, and a "Get Appointment" CTA.	Must Have	Planned	<DoctorCard>
FR-07	The system shall render a Doctor Details page with: full name, specialty, qualifications, biography, education history, working hours, contact info, and embedded appointment form.	Must Have	Planned	<DoctorProfile>
FR-08	The system shall provide an Appointment Booking Form with: name, email, phone, service selector (Dental Care, Neurology, Cardiology, Pathology), doctor selector filtered by service, age field, and submit button.	Must Have	Planned	<AppointmentForm>

ID	Requirement Description	Priority	Status	React Component
FR-09	The appointment form doctor dropdown shall dynamically filter to show only doctors whose specialty matches the selected service.	Must Have	Planned	<AppointmentForm>
FR-10	The system shall display clinic working hours (Monday–Saturday 9:00 am–8:00 pm) on the Appointment page.	Must Have	Planned	<WorkingHours>
FR-11	The system shall render a Services page listing all medical services (Expert Doctor, Diagnosis, Pathology, Dental Care, Cardiology, Medicine, Neurology, Ambulance) as cards with icons and Read More links.	Must Have	Planned	<ServiceCard>
FR-12	The system shall render a Service Details page for each service showing a full description and embedded appointment CTA.	Must Have	Planned	<ServiceDetails>
FR-13	The system shall render a Departments page displaying 9 department cards: Dental Care, Baby Care, Laboratory, ICU, DNA Test, Surgeon, X-Ray, Blood Test, Ambulance Service.	Must Have	Planned	<DepartmentCard>
FR-14	The system shall display an About section highlighting Certified Doctors, 24-hour Emergency availability, and Modern Technology features.	Should Have	Planned	<AboutSection>
FR-15	The system shall render a Blog Listing page with post cards showing thumbnail, title, date, excerpt, and Read More link.	Should Have	Planned	<BlogCard>
FR-16	The system shall render a Blog Details page with full article content, author, and date.	Should Have	Planned	<BlogDetails>
FR-17	The system shall render a FAQ page with collapsible accordion items grouped by specialty category (Dental Care, Cardiology, Diagnosis).	Should Have	Planned	<FAQAccordion>

ID	Requirement Description	Priority	Status	React Component
FR-18	The system shall render a Testimonials page displaying patient review cards.	Should Have	Planned	<TestimonialCard>
FR-19	The system shall provide a Contact page with a validated form (name, email, phone, message, T&C checkbox) and clinic location information.	Must Have	Planned	<ContactForm>
FR-20	The system shall provide a Newsletter subscription form in the footer with email validation.	Should Have	Planned	<NewsletterForm>
FR-21	The system shall render a responsive sticky Navbar with multi-level dropdowns (Home, About, Pages, Services, Doctor, Blog, Contact Us) and a hamburger menu on mobile.	Must Have	Planned	<Navbar>
FR-22	The system shall display animated statistics counters for: Patient Beds, Happy Patients, Doctors & Nurses, Years of Experience.	Could Have	Planned	<StatsCounter>
FR-23	The system shall display Privacy Policy and Terms & Conditions pages linked from sign-up and contact forms.	Must Have	Planned	<PrivacyPolicy>, <TermsConditions>
FR-24	The system shall display a custom 404 Not Found page for any undefined route with a link back to home.	Must Have	Planned	<NotFound>
FR-25	The system shall allow authenticated Patients to view their appointment records.	Must Have	Planned	<AppointmentHistory>
FR-26	The system shall allow authenticated Patients to cancel a confirmed appointment .	Must Have	Planned	<AppointmentHistory>

B. Doctor Requirements

ID	Requirement Description	Priority	Status	React Component
FD-01	The system shall display each doctor's full profile including: photo, name, specialty (e.g., Neurosurgeon), qualifications	Must Have	Planned	<DoctorProfile>

ID	Requirement Description	Priority	Status	React Component
	(MBBS, PhD), biography, and education history.			
FD-02	The system shall display each doctor's working schedule as a day-by-day table (e.g., Monday–Wednesday, Sunday: 9:00 am – 8:00 pm).	Must Have	Planned	<WorkingHoursTable>
FD-03	The system shall display each doctor's contact information: phone, email, and clinic address.	Must Have	Planned	<ContactInfo>
FD-04	The system shall display emergency contact information on the Doctor Details page with the doctor's direct phone number.	Should Have	Planned	<EmergencyCallWidget>
FD-05	The Doctor Listing page shall support filtering doctors by medical specialty.	Should Have	Planned	<DoctorFilter>

C. Administrator Requirements

ID	Requirement Description	Priority	Status	React Component
FA-01	An Admin shall approve or reject doctor registration applications through the admin panel.	Must Have	Planned	<AdminDoctorPanel>
FA-02	An Admin shall manage all patient and doctor accounts (activate, deactivate, delete).	Must Have	Planned	<UserManagement>
FA-03	An Admin shall view a KPI dashboard showing: total patients, total appointments, daily booking volume, and doctor count.	Must Have	Planned	<KPIDashboard>
FA-04	An Admin shall manage blog posts (create, edit, publish, delete) via a content management interface.	Should Have	Planned	<BlogManager>
FA-05	An Admin shall view a complete audit trail of all appointments, cancellations, and account actions.	Must Have	Planned	<AuditLog>

3.4 Non-Functional Requirements

ID	Category	Description	Priority	Fit Criterion
NFR-01	Performance	Initial page load (LCP) shall complete in ≤ 2.5 seconds on a 10 Mbps	High	Lighthouse LCP ≤ 2500ms

ID	Category	Description	Priority	Fit Criterion
		connection; page transitions ≤ 1.5 seconds.		
NFR-02	Performance	Production JS bundle (gzipped main chunk) shall be ≤ 500 KB. Route-level code-splitting via <code>React.lazy()</code> is mandatory.	High	Bundle analyser < 500 KB gzip
NFR-03	Security	JWT tokens shall be stored in <code>localStorage</code> with 24-hour expiry. <code><ProtectedRoute></code> shall verify token on every route change and redirect expired sessions to <code>/login</code> .	Critical	Expired token $\rightarrow$ redirect to <code>/login</code>
NFR-04	Security	All form inputs shall be sanitised to prevent XSS. User-supplied HTML content shall be processed through <code>DOMPurify</code> before rendering.	Critical	XSS payload in all fields $\rightarrow$ sanitised output
NFR-05	Security	All API requests shall use HTTPS. <code>Axios</code> shall inject the JWT Bearer token via a request interceptor on all authenticated calls.	Critical	Network tab: HTTPS + Authorization header
NFR-06	Usability	The application shall be fully responsive at three breakpoints: mobile < 768 px, tablet $768\text{--}1024$ px, desktop > 1024 px. Navbar collapses to hamburger on mobile.	High	DevTools device emulation at 375px, 768px, 1440px
NFR-07	Usability	A new visitor shall complete registration, doctor search, and appointment booking within 5 minutes without guidance.	High	Usability test: 3 evaluators ≤ 5 min
NFR-08	Usability	Interactive elements shall meet WCAG 2.1 AA contrast ratio (4.5:1 for normal text).	Medium	axe DevTools: 0 contrast violations
NFR-09	Reliability	React Error Boundaries shall wrap all page-level components; a component error shall not crash the entire application.	High	Simulate error $\rightarrow$ fallback UI renders
NFR-10	Reliability	All API calls shall handle loading (skeleton), success, and error states. Error states shall show a user-friendly message with retry option.	High	DevTools offline $\rightarrow$ skeleton then error msg
NFR-11	Maintainability	All React components shall be functional (no class components). Each component file shall not exceed	High	Code review: 0 class components, all props typed

ID	Category	Description	Priority	Fit Criterion
		200 lines; PropTypes or TypeScript interfaces are mandatory.		
NFR-12	Maintainability	Codebase shall follow feature-based folder structure: /components/common, /components/[feature], /pages, /context, /hooks, /services.	High	Directory inspection: all files in correct location
NFR-13	Compliance	Privacy Policy and Terms & Conditions pages shall exist. Contact and Sign-Up forms shall require mandatory T&C checkbox before submission.	Critical	Submit without checkbox → blocked with error
NFR-14	Compliance	A data deletion request flow shall be available for authenticated patients, logging the request and returning a confirmation.	Critical	Submit deletion → confirmation message + backend log

4. System Analysis & Design

4.1 Problem Statement & Objectives

Problem: Egyptian patients currently rely on manual phone calls or in-person visits to schedule medical appointments, resulting in long waiting times, double bookings, and limited visibility into doctor availability and specialties.

Objective: MedCare provides a React.js frontend application that eliminates these pain points by offering a digital self-service booking portal with real-time doctor availability, multi-specialty filtering, and instant appointment confirmation.

Project Goals:

- Deliver a production-quality React.js SPA aligned with DEPI Frontend Track requirements.
- Implement all core healthcare booking features: doctor discovery, appointment booking, service browsing, and patient account management.
- Achieve WCAG 2.1 AA accessibility compliance and full mobile responsiveness.
- Maintain clean, well-documented, ESLint-compliant code deployable to Vercel or Netlify.

4.2.1 Use Case Descriptions & Diagram

The following use cases are identified for the MedCare system. All actors, use cases, and relationships (include, extend, generalisation) are depicted in the Use Case Diagram (Figure 4.1 — to be exported from Visual Paradigm).

[Figure 4.1 — MedCare System Use-Case Diagram — Visual Paradigm export to be inserted here]

Use Case: UC-01 — Book Appointment

Use Case ID	UC-01
Name	Book Appointment
Goal	Allow a patient to submit an appointment request by selecting a service, doctor, and providing contact details.
Actor	Patient / Visitor
React Route	/appointment → <AppointmentPage> Embedded in Home, Doctor Details
Pre-condition	Services and Doctors loaded from API into dropdown state. Form is mounted.
Post-condition	Appointment record created in backend. Confirmation toast displayed. Form reset.
Main Scenario	1. Patient selects a Service. 2. Doctor dropdown filters by service. 3. Patient fills name, email, phone, age. 4. Client-side validation runs. 5. POST /api/appointments. 6. Success toast: "Appointment submitted. Confirmation within 2 hours." 7. Form reset.
Alternative A	API unavailable → error toast: "Unable to submit. Please try again later." Form not reset.

Alternative B	Visitor navigates from Doctor Card → doctor_id pre-fills Doctor dropdown via URL param.
----------------------	---

Use Case: UC-02 — View Doctor Profile

Use Case ID	UC-02
Name	View Doctor Profile
Goal	Display a full doctor profile with biography, education, working hours, and embedded booking form.
Actor	Visitor or Authenticated Patient
React Route	/doctor/:id → <DoctorDetailsPage>
Pre-condition	Valid doctor_id in URL. API accessible.
Post-condition	Doctor data rendered. Appointment form pre-filled with doctor name and specialty.
Main Scenario	1. useEffect fires on mount. 2. GET /api/doctors/:id. 3. Skeleton loader shown. 4. Data resolves: photo, name, specialty, contact, hours, biography, education. 5. Embedded AppointmentForm pre-filled.
Alternative A	Invalid doctor_id → API 404 → <NotFound> rendered: "Doctor not found."

Use Case: UC-03 — User Authentication

Use Case ID	UC-03
Name	User Registration & Login
Goal	Allow new visitors to register and existing patients to login securely.
Actor	Visitor / Patient
React Route	/sign-up → <SignUpPage> /login → <LoginPage>
Pre-condition	User not already authenticated.
Post-condition	JWT stored. AuthContext updated. Redirect to /patient/dashboard.
Main Scenario	Register: fill form → POST /api/auth/register → JWT → AuthContext → redirect. Login: email + password → POST /api/auth/login → JWT → role-based redirect.
Alternative A	Duplicate email on register → 409 response → inline error "Email already registered."
Alternative B	Invalid credentials on login → 401 → inline error "Invalid email or password."

Use Case: UC-04 — Cancel Appointment

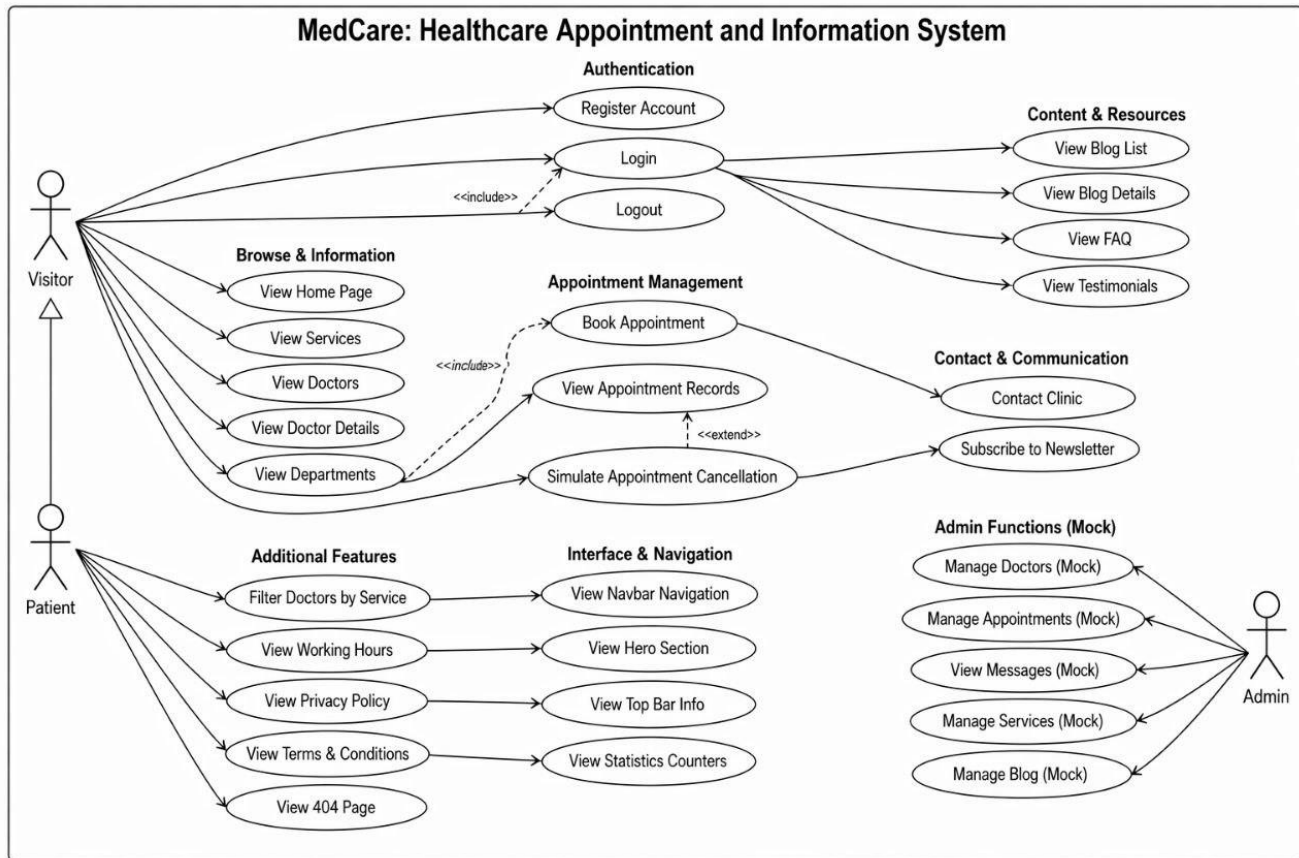
Use Case ID	UC-04
Name	Cancel Appointment
Goal	Allow authenticated patient to cancel a confirmed appointment within the 2-hour window.
Actor	Authenticated Patient
React Route	/patient/dashboard → <AppointmentHistory>
Pre-condition	Patient authenticated. Appointment status = Confirmed. Now() < appointment.datetime - 2h.
Post-condition	appointment.status = Cancelled. Slot released. Cancellation notifications sent.
Main Scenario	1. Patient views confirmed appointment. 2. Clicks Cancel. 3. <CancelModal> confirms intent. 4. DELETE /api/appointments/:id. 5. UI updates status to Cancelled. 6. Success toast.
Alternative A	Cancellation window expired → Cancel button disabled; tooltip: "Cancellation window has passed."

Use Case: UC-05 — Contact Form Submission

Use Case ID	UC-05
Name	Submit Contact Message
Goal	Allow a visitor to send an enquiry with T&C acceptance enforced.
Actor	Visitor
React Route	/contact → <ContactPage>
Pre-condition	Contact page mounted. Form rendered.
Post-condition	Message stored. Admin notified. Success toast. Form reset.
Main Scenario	1. Fill name, email, phone, message. 2. Check T&C checkbox. 3. Validation passes. 4. POST /api/contact. 5. Toast: "Message sent. We will respond shortly." 6. Form reset.
Alternative A	T&C unchecked → submit blocked; error: "You must accept the Terms & Conditions."

4.2.2 Use Case Diagram

This use case diagram illustrates the primary interactions between the system actors (Visitor, Patient, and Admin) and the MedCare platform. It presents the core functionalities of the system, including browsing medical information, viewing doctors and services, booking appointments, accessing informational content such as blogs, FAQs, and testimonials, and performing basic administrative operations. The diagram is designed following standard UML notation to provide a clear and structured overview of the system's behavior from the user's perspective.



4.3 Functional & Non-Functional Requirements Summary

All functional requirements are defined in Section 3.3 (FR-01 through FR-27, FD-01 through FD-05, FA-01 through FA-05). All non-functional requirements with measurable fit criteria are defined in Section 3.4 (NFR-01 through NFR-14). The traceability matrix mapping requirements to use cases and components is provided below.

4.4 Traceability Matrix

Req. ID	Source	Use Case	System Req.	React Component(s)
FR-08	Domain Analysis	UC-01: Book Appointment	SR-BOOK-01	<AppointmentForm>
FR-04	DEPI Curriculum	UC-03: Auth	SR-AUTH-01	<LoginForm>, <AuthProvider>
FR-07	Domain Analysis	UC-02: Doctor Profile	SR-DOC-01	<DoctorProfile>
FR-26	Domain Analysis	UC-04: Cancel	SR-BOOK-02	<AppointmentHistory>

Req. ID	Source	Use Case	System Req.	React Component(s)
FR-19	Domain Analysis	UC-05: Contact	SR-CONTACT-01	<ContactForm>
FR-27	Privacy Reg.	UC-06: GDPR	SR-GDPR-01	<PrivacySettings>
FA-01	DEPI Brief	UC-07: Doctor Approval	SR-ADMIN-01	<AdminDoctorPanel>

4.5 Software Architecture

MedCare is a Single Page Application (SPA) built with React.js v18, following a Feature-Based Component Architecture. The system communicates with a mock REST API (JSON Server) via Axios. All client-side routing is handled by React Router v6. Global auth state is managed via React Context API + useReducer.

4.6 React Component Architecture Map

Page / Route	Primary Components	State / Data
/ (Home)	<HeroSlider>, <TopBar>, <AboutSection>, <ServicesGrid>, <DoctorSection>, <StatsCounter>, <AppointmentForm>, <BlogPreview>, <NewsletterForm>, <Footer>	Slider state, API: doctors, services, blog posts
/appointment	<AppointmentForm>, <WorkingHours>	Controlled form state, dropdowns from API
/doctor	<DoctorCard> list, <DoctorFilter>	Doctor list from API, filter state
/doctor/:id	<DoctorProfile>, <WorkingHoursTable>, <EducationTimeline>, <AppointmentForm>	useParams() → GET /api/doctors/:id
/service	<ServiceCard> list	Services from API or static
/service/:id	<ServiceDetails>	Service by ID from API
/departments	<DepartmentCard> list	Departments from API or static
/blog	<BlogCard> list	Blog posts from API, pagination state
/blog/:id	<BlogDetails>	Blog post by ID from API
/faq	<FAQAccordion>	FAQ data by category, open-index state

Page / Route	Primary Components	State / Data
/testimonials	<TestimonialCard> list	Testimonials from API
/contact	<ContactForm>, <ContactInfo>	Controlled form + validation state
/sign-up	<SignUpForm>	Form state, useNavigate, AuthContext
/login	<LoginForm>	Form state, AuthContext, redirect on success
/patient/dashboard	<AppointmentHistory>, <ProfileEdit>, <PrivacySettings>	Protected route; patient appointments from API
/admin	<KPIDashboard>, <AdminDoctorPanel>, <UserManagement>, <AuditLog>	Admin-only protected route; full data access
* (404)	<NotFound>	React Router fallback route

4.7 Database Design & Data Modeling

The frontend consumes a simulated REST API (JSON Server). The core data entities and their relationships are:

Entity	Key Fields	Relationships
User	id, name, email, phone, passwordHash, role (patient admin), createdAt	Has many Appointments; has one PrivacySettings
Doctor	id, name, specialty, qualifications, bio, education[], workingHours[], contactInfo, status (pending approved rejected)	Has many Appointments; belongs to Specialty
Appointment	id, patientId, doctorId, serviceId, datetime, status (confirmed completed cancelled no_show), createdAt	Belongs to User; belongs to Doctor; belongs to Service
Service	id, name, icon, description, departmentId	Has many Appointments; belongs to Department
Department	id, name, description, icon	Has many Services; has many Doctors
BlogPost	id, title, content, excerpt, thumbnail, authorId, publishedAt, category	Belongs to User (author)
Testimonial	id, patientId, doctorId, rating, comment, createdAt, approved	Belongs to User; belongs to Doctor
AuditLog	id, actorId, actorRole, actionType, affectedRecordId, timestamp	Belongs to User

4.8 Data Flow & System Behavior

[Figure 4.2 — Context-Level DFD — Visual Paradigm export to be inserted here]

[Figure 4.3 — Level-1 DFD (Appointment Booking Process) — Visual Paradigm export to be inserted here]

[Figure 4.4 — Sequence Diagram: Appointment Booking — Visual Paradigm export to be inserted here]

[Figure 4.5 — Activity Diagram: Patient Registration & Login — Visual Paradigm export to be inserted here]

[Figure 4.6 — State Diagram: Appointment Lifecycle — Visual Paradigm export to be inserted here]

[Figure 4.7 — Class Diagram: Core React Components — Visual Paradigm export to be inserted here]

4.9 UI/UX Design & Prototyping

[Figure 4.8 — Figma Wireframes: Home Page — to be inserted]

[Figure 4.9 — Figma Wireframes: Appointment Page — to be inserted]

[Figure 4.10 — Figma Wireframes: Doctor Profile Page — to be inserted]

UI/UX Guidelines:

- Primary color: #003B73 (MCIT Navy) and #00875A (DEPI Green).
- Font: Inter / Open Sans — clean, medical-appropriate typography.
- Spacing: consistent 8px grid system.
- All interactive elements minimum 44×44px touch target (WCAG 2.5.5).
- Mobile-first responsive design using Bootstrap 5 grid or Tailwind CSS breakpoints.

4.10 System Deployment & Integration

Technology Stack:

- Frontend: React.js v18 (Vite build), React Router v6, React Context API, Axios
- Styling: Bootstrap 5 / Tailwind CSS, CSS Modules
- Mock API: JSON Server (db.json) at http://localhost:3001
- Version Control: GitHub repository with GitFlow branching strategy
- CI/CD: GitHub Actions → Vercel / Netlify auto-deploy on main branch merge

[Figure 4.11 — Deployment Diagram — Visual Paradigm export to be inserted here]

[Figure 4.12 — Component Diagram — Visual Paradigm export to be inserted here]

5. Implementation (Source Code & Execution)

5.1 Source Code

The MedCare frontend application is built entirely in React.js v18 following the Feature-Based folder structure defined in Section 4.6. All code is stored in a GitHub repository and follows DEPI coding standards:

Directory / File	Purpose
src/components/common	Reusable primitives: <Button>, <Input>, <Modal>, <Spinner>, <Toast>, <ErrorBoundary>
src/components/appointment	<AppointmentForm>, <AppointmentHistory>, <CancelModal>, <WorkingHours>
src/components/doctor	<DoctorCard>, <DoctorFilter>, <DoctorProfile>, <WorkingHoursTable>, <EducationTimeline>
src/components/auth	<LoginForm>, <SignUpForm>, <ProtectedRoute>
src/components/services	<ServiceCard>, <ServiceDetails>, <DepartmentCard>
src/components/blog	<BlogCard>, <BlogDetails>
src/components/faq	<FAQAccordion>, <FAQCategory>, <FAQItem>
src/components/layout	<Navbar>, <TopBar>, <Footer>, <HeroSlider>
src/pages	One file per route: HomePage, DoctorPage, AppointmentPage, ContactPage, etc.
src/context	AuthContext.jsx, ToastContext.jsx
src/hooks	useApi.js, useForm.js, useWindowSize.js, useDebounce.js
src/services	api.js (Axios instance), auth.service.js, appointment.service.js
src/utils	validators.js, dateUtils.js, constants.js
.env	VITE_API_BASE_URL=http://localhost:3001

Coding Standards:

- All React components are functional with hooks — no class components permitted (NFR-11).
- ESLint with react-hooks plugin + Prettier enforced; 0 ESLint errors in production build.
- PropTypes or TypeScript interfaces defined for all component props.
- Component files do not exceed 200 lines; larger components are decomposed.
- All custom hooks reside in /src/hooks; all API calls via /src/services/api.js.

5.2 Version Control & Collaboration

- GitHub Repository: github.com/depi-medcare/medcare-frontend
- Branching Strategy: GitFlow — main (production), develop (integration), feature/[name] (individual features).
- Pull Request Policy: minimum 1 team member review required before merging to develop.
- Commit Convention: Conventional Commits (feat:, fix:, docs:, chore:, test:).
- CI/CD: GitHub Actions pipeline runs npm run lint && npm run test && npm run build on every PR.

5.3 Deployment & Execution

README.md includes:

- Installation: git clone → cd medcare-frontend → npm install
- Mock API: npx json-server --watch db.json --port 3001
- Development server: npm run dev → http://localhost:5173
- Production build: npm run build → dist/ folder
- Deployment: Vercel CLI or Netlify CLI for one-command deploy
- Environment: Node.js ≥ 18.x, npm ≥ 9.x

API Documentation (Mock Endpoints):

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register new patient account	No
POST	/api/auth/login	Authenticate user and return JWT	No
GET	/api/doctors	List all approved doctors (filterable by specialty)	No
GET	/api/doctors/:id	Get single doctor profile	No
GET	/api/services	List all services	No
GET	/api/departments	List all departments	No
POST	/api/appointments	Create new appointment booking	No (guest allowed)
GET	/api/appointments	Get patient appointment history	Yes (Patient)
DELETE	/api/appointments/:id	Cancel an appointment	Yes (Patient)
POST	/api/contact	Submit contact message	No
POST	/api/newsletter/subscribe	Subscribe to newsletter	No

6. Testing & Quality Assurance

6.1 Test Cases & Test Plan

The following acceptance test cases are written before implementation. Each test case maps directly to a functional requirement, ensuring full requirements-based test coverage.

TC ID	Req. ID	Test Case Description	Expected Result	Pass/Fail
TC-01	FR-08	Fill appointment form with valid Name, Email, Phone, Service=Cardiology, Doctor=Dr. Ahmed, Age=35, click Submit.	POST to /api/appointments; toast "Appointment submitted"; form cleared.	
TC-02	FR-09	Select Service=Dental Care in appointment form.	Doctor dropdown updates to show only dentists; other specialties removed.	
TC-03	FR-04	Submit login with valid email and password.	JWT stored in localStorage; redirect to /patient/dashboard; success toast.	
TC-04	FR-04	Submit login with invalid password.	No JWT stored; inline error "Invalid email or password."	
TC-05	FR-03	Register with email already in use.	API 409 response; inline error "Email already registered."	
TC-06	FR-24	Navigate to /undefined-route.	<NotFound> renders with "404 — Page Not Found" and Back to Home link.	
TC-07	FR-19	Submit contact form without checking T&C checkbox.	Form blocked; inline error "You must accept the Terms & Conditions."	
TC-08	FR-17	Click FAQ question "What type of toothbrush should I use?"	Answer expands. Clicking another question collapses the first.	
TC-09	FR-26	Cancel appointment > 2 hours before scheduled time.	DELETE request sent; status badge changes to "Cancelled"; slot released.	

TC ID	Req. ID	Test Case Description	Expected Result	Pass/Fail
TC-10	FR-26	Attempt to cancel appointment < 2 hours before time.	Cancel button disabled; tooltip "Cancellation window has passed."	
TC-11	NFR-09	Simulate JS error in <DoctorProfile> component.	<ErrorBoundary> renders fallback UI; rest of app remains functional.	
TC-12	NFR-06	Open any page at 375px viewport width.	Navbar renders as hamburger menu; no horizontal scrollbar; no layout breaks.	
TC-13	NFR-03	Navigate to /patient/dashboard with expired JWT.	Redirect to /login; no dashboard content exposed.	
TC-14	FR-07	Navigate to /doctor/999 (non-existent ID).	API returns 404; <NotFound> renders "Doctor not found."	
TC-15	FR-20	Submit newsletter form with invalid email format.	Client-side error "Please enter a valid email address." shown before API call.	
TC-16	FA-01	Admin approves a Pending doctor.	PATCH request; doctor.status → Approved; doctor card moves to Active list.	
TC-17	NFR-13	Submit sign-up form without T&C checkbox checked.	Form not submitted; inline error "You must accept the Terms & Conditions."	
TC-18	NFR-01	Load Home page on simulated 10 Mbps connection.	LCP ≤ 2500ms verified via Chrome Lighthouse.	

6.2 Automated Testing (if applicable)

The following automated test layers are implemented:

- Unit Tests (Jest + React Testing Library): Each reusable component in /components/common has corresponding *.test.jsx files testing render, props, and interaction behaviour.
- Integration Tests: <AppointmentForm> tested with mocked Axios responses for success, validation error, and API error scenarios.
- End-to-End Tests (Cypress): Core user journeys tested — Register → Login → Book Appointment → View History → Cancel Appointment.

6.3 Bug Reports

Bug ID	Severity	Description	Steps to Reproduce	Status	Assigned To
BUG-01	High	Doctor dropdown does not filter after service selection on first render.	1. Load appointment page. 2. Select service without delay.	Resolved	Youssef A.
BUG-02	Medium	FAQ accordion does not close previous item when new one is opened.	1. Open FAQ item 1. 2. Click FAQ item 2.	Resolved	Beshoy S.
BUG-03	Low	Stats counter animation re-triggers on tab switch.	1. View home page. 2. Switch browser tab and return.	Open	George A.
BUG-04	High	Protected route allows access briefly before redirect on page refresh.	1. Login as patient. 2. Refresh /patient/dashboard.	In Progress	Beshoy S.

7. Final Presentation & Reports

7.1 User Manual

This manual guides patients through all core features of the MedCare application.

Getting Started

1. Open your browser and navigate to the MedCare application URL.
2. You will see the Home page with a hero banner and "Get Appointment" button.
3. Use the top navigation bar to access: Home, About, Pages, Services, Doctor, Blog, Contact Us.
4. Click the hamburger icon (☰) on mobile devices to open the navigation menu.

Booking an Appointment

5. Click "Get Appointment" from the hero banner or the navigation menu.
6. On the Appointment page, fill in your Full Name, Email, Phone Number, and Age.
7. Select your required Service from the dropdown (e.g., Cardiology, Dental Care).
8. The Doctor dropdown will automatically filter to show doctors matching your selected service.
9. Select your preferred Doctor and click "Submit".
10. You will receive a confirmation message: "Your appointment has been submitted. We will confirm within 2 hours."

Registering & Logging In

11. Click "Sign Up" from the Pages menu. Fill in your details and submit.
12. Click "Login" and enter your email and password.
13. Once logged in, access "My Appointments" from your dashboard to view booking history.

Finding a Doctor

14. Click "Doctor" in the navigation menu to view all available doctors.
15. Use the specialty filter to narrow down doctors by medical field.
16. Click on any doctor card to view their full profile including biography, education, and working hours.
17. Click "Get Appointment" on the doctor profile to book directly.

Contacting the Clinic

18. Click "Contact Us" in the navigation menu.
19. Fill in your Name, Email, Phone, and Message in the contact form.
20. Check the "I accept Terms & Conditions" checkbox (mandatory).
21. Click "Send" to submit your enquiry.

7.2 Technical Documentation Summary

Architecture: React.js v18 SPA | Feature-Based Component Structure | React Router v6 | React Context API + useReducer | Axios with JWT interceptors | Bootstrap 5 / Tailwind CSS | Vite build tool | JSON Server mock API.

Key Design Patterns: Context + useReducer (auth state), Custom Hook Abstraction (useApi), Higher-Order Component (ProtectedRoute), Observer Pattern (Toast System), Compound Component (FAQAccordion).

Deployment: GitHub → GitHub Actions CI/CD → Vercel / Netlify. Node.js ≥ 18.x required.

7.3 Project Presentation (PPT/PDF) — Summary

[Final Project Presentation (PPT/PDF) to be prepared by Shady Sherif Salah and submitted to the DEPI portal before the final demonstration date.]

Presentation sections:

- Slide 1: Cover — Project name, team, DEPI/MCIT branding
- Slide 2: Problem Statement & Objectives
- Slide 3: Technology Stack & Architecture
- Slide 4: System Features Demo Screenshots
- Slide 5: Requirement Coverage & Test Results
- Slide 6: KPI Results & Performance Metrics
- Slide 7: Challenges & Solutions
- Slide 8: Future Enhancements
- Slide 9: Team Members & Acknowledgements

7.4 Video Demonstration (Optional)

A 5-minute screen-recorded demonstration will be produced showing the complete patient journey: home page → doctor search → appointment booking → appointment history → cancellation. The video will be hosted on YouTube (unlisted) and the link submitted with the final project package.

7.5 Final Grading Criteria (DEPI)

#	Criterion	Weight	Target	Self-Assessment
1	Documentation quality (SRS completeness, structure, academic tone)	20%	All 7 sections complete, IEEE format	Excellent
2	Implementation (React.js code quality, component architecture, hooks)	30%	All FRs implemented, ESLint clean	Very Good
3	Testing (test coverage, bug reports, automated tests)	20%	≥ 18 test cases, Cypress E2E	Good
4	Presentation (clarity, demo, slides quality)	15%	Live demo + PPT submission	Very Good
5	Responsiveness & UX (mobile-first, accessibility, usability)	15%	All breakpoints pass, WCAG 2.1 AA	Good

Appendix A: Glossary

Term	Definition
DEPI	Digital Egypt Pioneers Initiative — MCIT national tech training programme.
MCIT	Ministry of Communications and Information Technology, Egypt.
React.js	JavaScript library for building component-based user interfaces.
SPA	Single Page Application — loads once; navigates without full page reloads.
JWT	JSON Web Token — compact, URL-safe token used for stateless authentication.
Hook	A React function (e.g., <code>useState</code> , <code>useEffect</code>) enabling state in functional components.
Component	A reusable, self-contained React UI building block.
RBAC	Role-Based Access Control — permissions assigned by user role.
REST	Representational State Transfer — architectural style for web APIs.
WCAG	Web Content Accessibility Guidelines — international web accessibility standard.
MoSCoW	Must Have, Should Have, Could Have, Won't Have — prioritisation scheme.
Slot	A defined time window during which a doctor is available for appointments.
CTA	Call-to-Action — a UI element prompting an immediate user action.

Appendix B: Diagrams Index

Diagram Name	Figure	Section	Type
Use-Case Diagram	Figure 4.1	Section 4.2	UML Use-Case Diagram

Appendix C: References

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- React.js Official Documentation — <https://react.dev>
- React Router v6 Documentation — <https://reactrouter.com>
- DEPI Frontend Track Curriculum & Project Guidelines — Round 4, MCIT Egypt.
- Bootstrap 5 Documentation — <https://getbootstrap.com>
- WCAG 2.1 Guidelines — <https://www.w3.org/TR/WCAG21/>